

ink! 3.0: Parity's Rust-Based Language for WASM Smart Contracts Gets a Major Update

ink!, Parity's Rust-based programming language for writing Web Assembly (Wasm) smart contracts, has just launched its biggest update to date...

Parity Technologies

March 15, 2022

5 Min Read



ink!, Parity's Rust-based programming language for writing Web Assembly (Wasm) smart contracts, has just launched its biggest update to date. ink! is used by developers in the Polkadot and Substrate ecosystem to build efficient, high-performance smart contracts and decentralized applications for Wasm virtual machines, which represent an alternative to Ethereum-based (EVM/Solidity) smart contract systems.

Substrate, the framework for building custom Polkadot-compatible as well as 'standalone' blockchains, contains modular building blocks for typical blockchain components that make it easier and faster for developers to build a blockchain. These building blocks are known as pallets. Substrate's library of pallets includes the Contracts pallet, which allows developers to build in Wasm-based smart contract functionality into their blockchain. [ink!](#), an extension of the Rust programming language, is what allows developers to write smart contracts for blockchains that utilize the Contracts pallet.

This latest version of ink! involved rewriting of all the major components that made up its predecessor, [ink! 2.0](#). During this time, many teams built Wasm smart contracts using ink!, and thanks in part to their contributions and feedback, the project has evolved significantly more than in previous releases.

Besides releasing ink! 3.0, we've also improved the Contracts pallet to boost parachain performance and added some new features for better cross-contract calls and performance, making it a lot easier for developers to write more complex logic. Finally, there are also some changes to the developer tooling with the release of cargo-contract 1.0.

Why Rust and Wasm?

At Parity we're huge fans of Rust, and have used it to write the initial implementation of Polkadot. Rust is an advanced and highly-efficient programming language that prioritizes

correctness and helps prevent software bugs, with a robust community around it and support from major companies like Mozilla, Google, Microsoft, and Amazon.

Using Rust as the basis for ink! provides us with the correctness guarantees of the strict Rust compiler, and it lets us tap into all the improvements happening in the Rust ecosystem for free. These improvements can range from better optimizations in the compiler, to useful third party libraries. Learn more about why Rust is well suited for smart contracts [here](#).

Wasm is another technology that plays a key role in the design for Polkadot for several reasons. Wasm is fast, safe, lightweight, high-performance, and efficient to execute, being built as close to machine code as possible. It is executable in web browsers, thus facilitating innovation in decentralized applications designed for user experience and widespread adoption. Wasm allows us to take advantage of its mainstream adoption, as it also enjoys a high level of use by major companies and has a huge community of developers. This also allows us to reap all the benefits of compiler improvements and a whole ecosystem of tooling which will only grow over time.

Wasm expands the number of languages that smart contracts can be written in, including Go, C/C++, and of course Rust. For all the benefits it provides, we believe that Wasm represents the future of smart-contract infrastructure and applications and represents an improvement over previous generation systems.

Learn more about why Wasm is well suited for smart contracts [here](#).

ink! 3.0

The main idea behind ink! 3.0 is that it looks and feels just like Rust. ink! syntax has changed slightly to better map from ink! to the generated Rust code. What users see is mostly what will be generated by ink!. Now, the syntax is pretty much identical to the Rust programming language. This has the advantage that all the tooling Rust developers know and love – such as rust-analyzer and Clippy – just works out of the box.

Besides feeling like Rust, let's take a look at the most significant changes that came to ink! with version 3.0.

-Trait Support --- One of the most anticipated features of ink! 3.0 is its Rust trait support best described as providing a more Rusty development experience. Through the new `# [ink::trait_definition]` procedural macro it is now possible to define your very own trait definitions that are then implementable by ink! smart contracts. This allows defining shared smart contract interfaces to different concrete implementations. This can be useful

for defining community defined contract standards. Note that this ink! trait definition can be defined anywhere, even in another crate!

-Reduced contract sizes --- Smaller contract sizes mean better performance and less gas costs. The previous ink! generated smart contracts were big in Wasm code size, which lead to parachain transaction throughput being limited by bandwidth rather than computational complexity. Among the many solutions considered to reduce the size of smart contracts was to reduce the size of the compiled contracts.

-Delegate Call --- The addition of `delegate_call` allows a way for contracts to forward a call they received to another contract. One use case for this is implementing a Proxy Upgrade Pattern for smart contracts.

-Chain Extensions - This feature allows developers to call directly into Substrate pallets from their contracts, allowing them to tap into the rich ecosystem of Substrate development.

cargo-contract 1.0

Alongside the release of ink! 3.0 we're also releasing cargo-contract 1.0. cargo-contract is a handy developer tool for building, testing, and deploying ink! contracts. Some notable updates in this release include:

-New CLI commands for interacting with contracts on-chain: upload, instantiate and call! This means users are no longer required to click through a browser UI to interact with their contract on a chain, they can also use the command-line now. This also paves the way for scripting and integrations into Continuous Integration environments.

-Linting rules for ink! smart contracts --- Through automated checking of source code for programmatic and stylistic errors, ink! developers get warnings about common errors. Going forward we want to successively build the collection of linting rules for ink! contracts out. Things we have in mind are rules guiding you to reduce gas costs, towards small contract footprints, correct API usage patterns, and pointing out security flaws.

Substrate's Contracts pallet

The Contracts pallet is Substrate's execution environment for Wasm smart contracts. It can execute smart contracts in any programming language that compiles down to Wasm, as long as the Contracts pallet API is implemented. ink! Is one language which does this for Rust.

Prior versions of the pallet used state rent as a mechanism to cope with state bloat. In the latest iteration state rent was replaced by automatic deposit collection. Previously, a contract needed to make sure to always have enough balance to pay for its storage. Otherwise, it was put in a disabled state until someone revived it. This added a lot of additional work and consideration to the process of contract writing.

Under the regime which we call "automatic deposit collection" this cost is automatically paid by the caller of a contract rather than the contract itself. A caller of a contract pays a deposit to each contract in which new storage was created as a result of the executed call. In a similar vein, a caller gets a refund from all the contracts that the call removed storage from.

This new system completely removes the possibility of contracts being set into an inactive state. However, contract authors still want to use as little storage as possible and create ways to remove storage when it is no longer needed in order to keep the costs of using the contract down.

Thanks to the ink! community

Over many years and several releases, ink! has evolved to hit the target of looking and feeling like Rust. At this stage, it has all the features Rust programmers recognize and need to further enhance their ability to write, update, and reason about complex smart contracts. While ink! is now a comprehensive language, it's still a language in development, and above all, a community-focused project.

Thanks to the input of the ink! community, ink! has developed to become a language that satisfies the needs of our shared future.

Follow what's happening

So, what are you waiting for? Check out what's going on, and see what you like about ink!.

For developers interested in learning more about building Wasm smart contracts, check out the [first ever Wasm smart contracts conference](#), coming up on May 31, 2022 and organized by the community.

You can find regular updates happening under the Parity Smart Contracts umbrella [here](#), and see the ink! Repo [here](#). You can also see a [guided tutorial for first steps with ink!](#), see the in-depth ink! [documentation](#), and watch some live presentations about ink! ([video 1](#), [video 2](#)).

Learn more about developing in the Substrate ecosystem at substrate.dev, sign up for the [Substrate Developer Newsletter](#), join the [Substrate Seminar community calls](#), and follow Substrate on [Twitter](#).



Company

Technology

Culture

Careers

About Us

Blog

Contact

Opportunities

Parity Data Dashboard

GitHub

Bug Bounty

Blockchain Academy